

```
// =====
// scores/sofa.ts
// -----
// SOFA Score refatorado com TODOS os bugs P0 da auditoria corrigidos:
//
// Bug #1 ✓ SOFA Cardio: usa calcDoseInfusao real + peso + diluição
// Bug #2 ✓ SOFA Neuro: suprimido quando sedação ativa (flag sobSedacao)
// Bug #5 ✓ SOFA Resp: SOFA 3 e 4 exigem VM ativa (não só P/F ratio)
// Bug #6 ✓ SOFA Renal: cross-reference com débito urinário e TRRC
// Bug #7 ✓ parseFloatBR em todas as leituras numéricas
// Bug #9 ✓ coerceFiO2Input para P/F correto
// Bug #11 ✓ pam1 é MIN (valor pior)
//
// Funções de componente exportadas individualmente para testes unitários.
// Singer M et al., JAMA 2016. DOI:10.1001/jama.2016.0287
// =====

import type { Patient, Infusao } from '../types';
import type { SOFAResult, SOFAComponents } from '../types/clinical';
import { DVA_DICT } from '../dictionaries';
import { parseFloatBR } from '../calculations/parseBR';
import { calcDoseInfusao, isVasopressorHighDose } from '../calculations/infusao';
import { calcDiureseEfetiva } from '../calculations/diurese';
import { coerceFiO2Input, coercePlaquetas } from '../guards/unitCoercion';
import { HEMO, RENAL, NEURO } from '../constants';

// =====
// Função principal
// =====

/**
 * Calcula SOFA Score completo a partir de um objeto Patient.
 * Retorna SOFAResult com total, detalhamento por componente e dados faltantes.
 */
export function getSOFA(patient: Patient): SOFAResult {
  const comps: SOFAComponents = {
    resp: 0,
    coag: 0,
    liver: 0,
    cardio: 0,
    neuro: 0,
    renal: 0,
  };

  const detail: string[] = [];
  const missing: string[] = [];
  const suppressed: string[] = [];
  const warnings: string[] = [];

  // — 1. RESPIRATÓRIO —————
  const respComp = sofaResp(
    patient.resp.pao2,
    patient.resp.vmFio2 || patient.resp.fio202,
    patient.resp.suporte === 'IOT + VM',
  );
  comps.resp = respComp.score;
  if (respComp.detail) detail.push(respComp.detail);
  if (respComp.missing) missing.push(respComp.missing);
  if (respComp.warning) warnings.push(respComp.warning);

  // — 2. COAGULAÇÃO —————
  const coagComp = sofaCoag(patient.hemato.plaq1, patient.hemato.plaqUnit);
  comps.coag = coagComp.score;
```

```

if (coagComp.detail) detail.push(coagComp.detail);
if (coagComp.missing) missing.push(coagComp.missing);
if (coagComp.warning) warnings.push(coagComp.warning);

// — 3. HEPÁTICO —————
const liverComp = sofaLiver(patient.tgi.bb);
comps.liver = liverComp.score;
if (liverComp.detail) detail.push(liverComp.detail);
if (liverComp.missing) missing.push(liverComp.missing);

// — 4. CARDIOVASCULAR —————
const cardioComp = sofaCardio(
  patient.hemo.pam1, // pam1 = valor MIN (pior PAM registrada) — Bug #11
  patient.dvas,
  patient.peso,
);
comps.cardio = cardioComp.score;
if (cardioComp.detail) detail.push(cardioComp.detail);
if (cardioComp.missing) missing.push(cardioComp.missing);

// — 5. NEUROLÓGICO —————
const gcsEscala = patient.neuro.escalas.find(e => e.nome === 'ECG (Glasgow)');
const gcsValue = gcsEscala?.valor ?? '';
const sedacaoAtiva = patient.sedativos.length > 0;

// Bug #2: se paciente está sob sedação, GCS não reflete função neurológica
const sobSedacao = gcsEscala?.sobSedacao ?? sedacaoAtiva;

const neuroComp = sofaNeuro(gcsValue, sobSedacao, gcsEscala?.preSedationValue);
comps.neuro = neuroComp.score;
if (neuroComp.detail) detail.push(neuroComp.detail);
if (neuroComp.missing) missing.push(neuroComp.missing);
if (neuroComp.suppressed) suppressed.push(neuroComp.suppressed);

// — 6. RENAL —————
const renalComp = sofaRenal(
  patient.renal.cr1,
  patient.renal.diurese,
  patient.peso,
  patient.renal.diureseHoras,
  patient.renal.trrc,
);
comps.renal = renalComp.score;
if (renalComp.detail) detail.push(renalComp.detail);
if (renalComp.missing) missing.push(renalComp.missing);
if (renalComp.warning) warnings.push(renalComp.warning);

const total = Object.values(comps).reduce((a, b) => a + b, 0);

return {
  total,
  components: comps,
  detail,
  missing,
  suppressed,
  warnings,
  computedAt: new Date().toISOString(),
};
}

// =====
// Componentes individuais (exportados para testes unitários)
// =====

interface ComponentResult {
  score: number;

```

```

    detail?: string;
    missing?: string;
    suppressed?: string;
    warning?: string;
  }

// — Respiratório —————
/**
 * Bug #5 CORRIGIDO: SOFA Resp 3 e 4 exigem VM ativa.
 * Sem VM, P/F < 200 é SOFA 2, não 3.
 *
 * Referência: Singer 2016 – Tabela 2.
 */
export function sofaResp(
  pao2Raw: string,
  fio2Raw: string,
  onVM: boolean,
): ComponentResult {
  const pao2 = parseFloatBR(pao2Raw);

  if (isNaN(pao2) || pao2 <= 0) {
    return { score: 0, missing: 'PaO2' };
  }

  const { value: fio2 } = coerceFiO2Input(fio2Raw || '21');
  // Bug #9: fio2 agora sempre em 21-100 percentual, nunca 0.21

  if (fio2 <= 0) {
    return { score: 0, missing: 'FiO2' };
  }

  // P/F = (PaO2 / FiO2%) × 100
  const ratio = (pao2 / fio2) * 100;
  const ratioStr = Math.round(ratio).toString();

  let score = 0;
  let warning: string | undefined;

  if (ratio >= 400) {
    score = 0;
  } else if (ratio >= 300) {
    score = 1;
  } else if (ratio >= 200) {
    score = 2;
  } else if (ratio >= 100) {
    // Bug #5: SOFA 3 APENAS com VM
    score = onVM ? 3 : 2;
    if (!onVM) {
      warning = `P/F ${ratioStr} seria SOFA Resp 3 mas paciente não está em VM
invasiva – pontuando 2`;
    }
  } else {
    // ratio < 100
    // Bug #5: SOFA 4 APENAS com VM
    score = onVM ? 4 : 3;
    if (!onVM) {
      warning = `P/F ${ratioStr} seria SOFA Resp 4 mas paciente não está em VM
invasiva – pontuando 3`;
    }
  }

  const vmLabel = onVM ? ' + VM' : '';
  return {
    score,
    detail: `Resp: ${score} (P/F ${ratioStr}${vmLabel})`,
    warning,
  };
}

```

```

    };
}

// — Coagulação —————
/**
 * Bug #8 CORRIGIDO: coercePlaquetas normaliza unidade antes de pontuar.
 */
export function sofaCoag(
  plaqRaw: string,
  plaqUnit?: 'x103/μL' | '/μL',
): ComponentResult {
  const coerced = coercePlaquetas(plaqRaw, plaqUnit);

  if (coerced.value === null) {
    return { score: 0, missing: 'Plaquetas' };
  }

  const plaq = coerced.value; // sempre em x103/μL
  let score = 0;

  if (plaq < 20)      score = 4;
  else if (plaq < 50) score = 3;
  else if (plaq < 100) score = 2;
  else if (plaq < 150) score = 1;

  const unitWarning = coerced.warning === 'ambiguous_input'
    ? '⚠ unidade plaquetas ambígua'
    : '';

  return {
    score,
    detail: score > 0 ? `Coag: ${score} (Pla q ${plaq}x103${unitWarning})` : undefined,
    warning: coerced.warning === 'ambiguous_input'
      ? `Unidade de plaquetas ambígua (${coerced.rawInput}) – confirmar se é /μL ou
x103/μL`
      : undefined,
  };
}

// — Hepático —————
export function sofaLiver(bbRaw: string): ComponentResult {
  const bb = parseFloatBR(bbRaw);

  if (isNaN(bb) || bb <= 0) {
    return { score: 0, missing: 'Bilirrubina' };
  }

  let score = 0;
  if (bb >= 12.0)      score = 4;
  else if (bb >= 6.0)  score = 3;
  else if (bb >= 2.0)  score = 2;
  else if (bb >= 1.2)  score = 1;

  return {
    score,
    detail: score > 0 ? `Hep: ${score} (BB ${bb} mg/dL)` : undefined,
  };
}

// — Cardiovascular —————
/**
 * Bug #1 CORRIGIDO: usa calcDoseInfusao real (com peso + diluição) para
 * determinar se vasopressor está em alta dose (>0.1 mcg/kg/min Nor/Epi).
 *
 * Critério SOFA Cardio (Singer 2016):
 * 0 = PAM ≥ 70

```

```

* 1 = PAM < 70 sem DVA
* 2 = Dopamina ≤ 5 OU Dobutamina qualquer dose
* 3 = Dopamina > 5 OU Noradrenalina/Epinefrina ≤ 0.1 mcg/kg/min
* 4 = Dopamina > 15 OU Noradrenalina/Epinefrina > 0.1 mcg/kg/min
*/
export function sofaCardio(
  pamMinRaw: string,
  dvas: Infusao[],
  peso: string,
): ComponentResult {
  if (!dvas || dvas.length === 0) {
    // Sem DVA – avaliar PAM
    const pam = parseFloatBR(pamMinRaw);
    if (isNaN(pam)) {
      return { score: 0, missing: 'PAM' };
    }
    if (pam < HEMO.PAM_SEPSE_TARGET) {
      return {
        score: 1,
        detail: `Cardio: 1 (PAM ${pam} mmHg, sem DVA)`,
      };
    }
    return { score: 0 };
  }

  // COM DVA – calcular doses reais
  let maxScore = 2; // DVA presente = mínimo SOFA 2
  const dvaDetails: string[] = [];

  for (const dva of dvas) {
    if (!dva.droga) continue;

    const dict = DVA_DICT[dva.droga];
    if (!dict) continue;

    const doseResult = calcDoseInfusao(dva, dict, peso);

    if (doseResult.error === 'missing_weight') {
      // Sem peso, não dá pra calcular – assume score 3 conservador
      maxScore = Math.max(maxScore, 3);
      dvaDetails.push(`${dva.droga} (sem peso – assumindo SOFA 3)`);
      continue;
    }

    if (!doseResult.value || isNaN(doseResult.value)) {
      dvaDetails.push(`${dva.droga} (vazão inválida)`);
      continue;
    }

    const dose = doseResult.value;
    const unit = doseResult.unit;
    const doseStr = `${dose.toFixed(3)} ${unit}`;

    if (dva.droga === 'Dobutamina') {
      // Dobutamina = SOFA 2 independente da dose
      maxScore = Math.max(maxScore, 2);
      dvaDetails.push(`Dobuta ${doseStr}`);
      continue;
    }

    if (dva.droga === 'Vasopressina') {
      // Vasopressina adjuvante = equivalente a SOFA 3 (não tem cutoff oficial)
      maxScore = Math.max(maxScore, 3);
      dvaDetails.push(`Vasopressina ${doseStr}`);
      continue;
    }
  }
}

```

```

    if (dva.droga === 'Dopamina') {
      if (dose > 15) {
        maxScore = Math.max(maxScore, 4);
        dvaDetails.push(`Dopa ${doseStr} □□`);
      } else if (dose > 5) {
        maxScore = Math.max(maxScore, 3);
        dvaDetails.push(`Dopa ${doseStr}`);
      } else {
        maxScore = Math.max(maxScore, 2);
        dvaDetails.push(`Dopa ${doseStr}`);
      }
      continue;
    }

    // Noradrenalina / Epinefrina – Bug #1 fix principal
    if (dva.droga === 'Noradrenalina' || dva.droga === 'Adrenalina') {
      const isHighDose = isVasopressorHighDose(doseResult, dva.droga);
      if (isHighDose) {
        maxScore = Math.max(maxScore, 4);
        dvaDetails.push(`Nor/Epi ${doseStr} > 0.1 □□`);
      } else {
        maxScore = Math.max(maxScore, 3);
        dvaDetails.push(`Nor/Epi ${doseStr}`);
      }
      continue;
    }

    // Outros vasopressores (Nipride, Tridil, Esmolol) = SOFA 2
    maxScore = Math.max(maxScore, 2);
    dvaDetails.push(`${dva.droga} ${doseStr}`);
  }

  return {
    score: maxScore,
    detail: `Cardio: ${maxScore} (${dvaDetails.join('; ')} )`,
  };
}

// ————— Neurológico —————
/**
 * Bug #2 CORRIGIDO: não pontua se paciente está sob sedação.
 * Se sobSedacao = true e há preSedationValue, usa o valor pré-sedação.
 */
export function sofaNeuro(
  gcsRaw: string,
  underSedation: boolean,
  preSedationGCS?: string,
): ComponentResult {
  // Se sob sedação E sem GCS pré-sedação documentado → suprimir componente
  if (underSedation && !preSedationGCS) {
    return {
      score: 0,
      suppressed: 'Neuro suprimido: paciente sob sedação ativa (GCS reflete droga, não lesão). Documentar GCS pré-sedação para pontuação.',
    };
  }

  // Usar GCS pré-sedação se disponível e paciente está sedado
  const effectiveGCS = (underSedation && preSedationGCS) ? preSedationGCS : gcsRaw;
  const gcs = parseInt(String(effectiveGCS).replace(/^[^0-9]/g, ''), 10);

  if (isNaN(gcs) || gcs < 3 || gcs > 15) {
    return { score: 0, missing: 'Glasgow' };
  }

  let score = 0;

```

```

    if (gcs < 6)          score = 4;
    else if (gcs <= 9)    score = 3;
    else if (gcs <= 12)  score = 2;
    else if (gcs <= 14)  score = 1;

    const sedLabel = underSedation ? ' (pré-sedação)' : '';

    return {
      score,
      detail: score > 0 ? `Neuro: ${score} (GCS ${gcs}${sedLabel})` : undefined,
    };
  }
}

// — Renal —————
/**
 * Bug #6 CORRIGIDO: cross-reference creatinina + débito urinário + TRRC.
 * Score = MAX(scorePorCr, scorePorDU, trrcScore).
 *
 * KDIGO 2012 + SOFA Singer 2016.
 */
export function sofaRenal(
  crRaw: string,
  diureseRaw: string,
  pesoRaw: string,
  horasRaw: string,
  trrc?: boolean,
): ComponentResult {
  // TRRC → SOFA 4 automático
  if (trrc) {
    return {
      score: 4,
      detail: 'Renal: 4 (TRRC ativa)',
    };
  }

  const details: string[] = [];

  // Score por Creatinina
  let scoreByCr = 0;
  const cr = parseFloatBR(crRaw);
  if (!isNaN(cr) && cr > 0) {
    if (cr >= 5.0)      scoreByCr = 4;
    else if (cr >= 3.5) scoreByCr = 3;
    else if (cr >= 2.0) scoreByCr = 2;
    else if (cr >= 1.2) scoreByCr = 1;
    details.push(`Cr ${cr} → SOFA ${scoreByCr}`);
  } else {
    details.push('Cr: sem dado');
  }

  // Score por Débito Urinário (Bug #6 fix)
  let scoreByDU = 0;
  const duResult = calcDiureseEfetiva(diureseRaw, pesoRaw, horasRaw);
  if (duResult.mlPerKgPerHour !== null) {
    scoreByDU = duResult.kdigoByDU >= 3 ? 4 :
      duResult.kdigoByDU === 2 ? 3 :
      duResult.kdigoByDU === 1 ? 1 : 0;
    // KDIGO 1 por DU → SOFA 1 | KDIGO 2 → SOFA 3 | KDIGO 3 → SOFA 4
    details.push(`DU ${duResult.formatted} ml/kg/h → ${duResult.kdigoLabel}`);
  }

  // Score final = pior dos dois
  const finalScore = Math.max(scoreByCr, scoreByDU) as 0 | 1 | 2 | 3 | 4;

  const missing: string[] = [];
  if (isNaN(cr)) missing.push('Creatinina');

```

```

    if (duResult.mlPerKgPerHour === null) missing.push('Débito urinário');

    const warning = !isNaN(cr) && duResult.mlPerKgPerHour !== null && scoreByDU >
scoreByCr
    ? `DU oligúrico eleva SOFA Renal de ${scoreByCr} para ${finalScore} – IRA por
débito!`
    : undefined;

    return {
      score: finalScore,
      detail: `Renal: ${finalScore} (${details.join('; ')}`,
      missing: missing.length > 0 ? missing.join(', ') : undefined,
      warning,
    };
  }
}

```